SOLID Principles

Theory + Interview Questions & Answers

Java edition: Basic, Intermediate, Advanced, Spring Boot scenarios and coding challenges

github.com/mateenms/system-design

Part 1: Theory

> "The goal of the SOLID principles is the creation of mid-level software structures that are **tolerant of change**, **easy to understand**, and the **basis of components that can be reused** in many software systems." — Robert C. Martin

---

1. What is SOLID?

SOLID is an acronym coined by **Michael Feathers** from five principles introduced and popularised by **Robert C. Martin ("Uncle Bob")** in the early 2000s. They are design guidelines for **object-oriented** systems, and Java — being a strictly OO language — is where they are most naturally expressed.

Letter	Principle	Author's One-Liner
S	Single Responsibility	A class should have one, and only one, reason to change.
O	Open/Closed	Open for extension, closed for modification.
L	Liskov Substitution	Subtypes must be substitutable for their base types.
I	Interface Segregation	Many client-specific interfaces are better than one general-purpose interface.
D	Dependency Inversion	Depend upon abstractions, not concretions.

---

2. Why SOLID Matters

Without these principles, large codebases tend toward:

- **Rigidity** — a small change requires modifying many classes
- **Fragility** — unrelated modules break when you touch something
- **Immobility** — hard to extract and reuse components in other projects
- **Viscosity** — it is easier to hack than to do it right

Applying SOLID leads to:

- Easier unit testing (each unit has a single, clear job)
- Easier extension (add new behaviour without touching old code)
- Safer refactoring (change is local, not global)
- Better readability (small, focused classes and interfaces)

---

3. S — Single Responsibility Principle

Core Idea

> A class should have **one, and only one, reason to change**.

"Reason to change" maps to a **stakeholder or actor**. If Finance, HR, and the DBA can all independently force a class to change, that class has three responsibilities.

Mental Model

Think of it as **cohesion**. Group things that change together; separate things that change for different reasons.

Bad Example

```
// BAD — three actors can force this class to change
public class Employee {

    // Finance team controls this
    public double calculatePay() {
        return baseSalary * 1.2;
    }

    // DBA controls this (schema changes)
    public void save() {
        // JDBC logic
    }

    // HR team controls this
    public String generateReport() {
        return "Employee Report: " + name;
    }
}
```

Good Example

```
// GOOD — one reason to change per class

public class Employee {
    private String name;
    private double baseSalary;
    // getters/setters
}

public class PayCalculator {
    public double calculatePay(Employee employee) {
        return employee.getBaseSalary() * 1.2;
    }
}

public class EmployeeRepository {
    public void save(Employee employee) {
        // JDBC logic — only changes when DB schema changes
    }
}

public class EmployeeReporter {
    public String generateReport(Employee employee) {
        return "Employee Report: " + employee.getName();
    }
}
```

Real-World Application

- **Spring Boot**: `@Service` for business logic, `@Repository` for DB, `@Controller` for HTTP — each has one responsibility
- **Microservices**: SRP at service level — each service owns one bounded context
- **Utility classes**: a `StringUtils` class doing 40 things is an SRP smell

Signs You Are Violating SRP

- Class name contains "Manager", "Handler", "Helper", "Util", "And"
- The class imports from completely unrelated packages (e.g., both `java.sql` and `javax.mail`)
- Changing a DB column forces you to update email formatting logic

---

4. O — Open/Closed Principle

Core Idea

> Software entities should be **open for extension** but **closed for modification**.

Add new behaviour by **adding new code**, not by editing existing, tested code.

Mental Model

Think of a **plugin architecture**. Adding a plugin does not modify the core application. The enabling mechanism is **abstraction** — define a stable interface and extend it.

Bad Example

```
// BAD — every new discount type requires editing applyDiscount()
public class DiscountService {
    public double applyDiscount(Order order, String discountType) {
        if (discountType.equals("PERCENTAGE")) {
            return order.getTotal() * 0.9;
        } else if (discountType.equals("FIXED")) {
            return order.getTotal() - 10;
        }
        // adding SEASONAL requires editing this method
        return order.getTotal();
    }
}
```

Good Example

```
// GOOD — new discount types extend without modifying DiscountService

public interface DiscountStrategy {
    double apply(double total);
}

public class PercentageDiscount implements DiscountStrategy {
    @Override
    public double apply(double total) { return total * 0.9; }
}

public class FixedDiscount implements DiscountStrategy {
    @Override
    public double apply(double total) { return total - 10; }
}

public class SeasonalDiscount implements DiscountStrategy { // new — zero changes to existing
code
    @Override
    public double apply(double total) { return total * 0.75; }
}

public class DiscountService {
    public double applyDiscount(Order order, DiscountStrategy strategy) {
        return strategy.apply(order.getTotal()); // never changes
    }
}
```

Design Patterns That Implement OCP

Pattern	How
Strategy	Swap algorithms behind a stable interface
Decorator	Add behaviour without modifying the decorated class
Template Method	Subclasses override steps; skeleton is fixed
Observer	Add new listeners without touching the publisher

Signs You Are Violating OCP

- Long `if/else if` or `switch` blocks dispatching on a type string/enum
- Adding a new "type" requires editing 5+ existing files
- Business logic contains `instanceof` checks

---

5. L — Liskov Substitution Principle

Core Idea

> If `S` is a subtype of `T`, then objects of type `T` may be replaced with objects of type `S` **without altering the correctness of the program**.

Anywhere you use the parent class, you must be able to use the child class without surprises.

Formal Contract Rules (Barbara Liskov, 1987)

1. **Preconditions** cannot be strengthened in the subtype
2. **Postconditions** cannot be weakened in the subtype
3. **Invariants** of the supertype must be preserved
4. The subtype must not throw new checked exceptions not declared by the base

Classic Violation: Rectangle / Square

```
// BAD — Square breaks Rectangle's implicit contract
public class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width)   { this.width = width; }
    public void setHeight(int height) { this.height = height; }
    public int area() { return width * height; }
}

public class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width;  // side-effect violates caller's expectation
    }
    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
}

// Code written for Rectangle breaks when handed a Square:
public void resize(Rectangle r) {
    r.setWidth(5);
    r.setHeight(10);
    assert r.area() == 50; // FAILS for Square — area is 100
}
```

Good Example

```
// GOOD — share only a true common abstraction
public interface Shape {
    int area();
}

public class Rectangle implements Shape {
    private int width, height;
    public Rectangle(int w, int h) { this.width = w; this.height = h; }
    @Override public int area() { return width * height; }
}

public class Square implements Shape {
    private int side;
    public Square(int side) { this.side = side; }
    @Override public int area() { return side * side; }
}
```

Real-World Application

- **Spring Data**: `JpaRepository` extends `CrudRepository` — every CRUD method must work correctly on the child
- **Collection API**: `UnmodifiableList` that throws on `add()` — callers expecting a mutable `List` break
- **Test doubles**: a mock `PaymentGateway` must behave like the real one in all respects callers rely on

Signs You Are Violating LSP

- A subclass method throws `UnsupportedOperationException`
- Business logic uses `instanceof` to check the concrete type before calling a method
- Tests for the parent class fail when run against the subclass

---

6. I — Interface Segregation Principle

Core Idea

> No client should be forced to **depend on methods it does not use**.

Prefer many small, role-specific interfaces over one large general-purpose interface.

Mental Model

Think of a **TV remote vs a universal remote**. A universal remote has 80 buttons nobody uses. A role-specific interface exposes only what its client needs.

Bad Example

```
// BAD — every device must implement every method
public interface IMultiFunctionDevice {
    void print(Document doc);
    void scan(Document doc);
    void fax(Document doc);       // most printers cannot fax
    void email(Document doc);     // most printers cannot email
}

public class BasicPrinter implements IMultiFunctionDevice {
    @Override public void print(Document doc) { /* real logic */ }
    @Override public void scan(Document doc)  { /* real logic */ }
    @Override public void fax(Document doc)   { throw new UnsupportedOperationException(); }
    @Override public void email(Document doc) { throw new UnsupportedOperationException(); }
}
```

Good Example

```
// GOOD — segregated interfaces
public interface IPrintable  { void print(Document doc); }
public interface IScannable  { void scan(Document doc); }
public interface IFaxable    { void fax(Document doc); }
public interface IEmailable  { void email(Document doc); }

public class BasicPrinter implements IPrintable {
    @Override public void print(Document doc) { /* ... */ }
}

public class ScannerPrinter implements IPrintable, IScannable {
    @Override public void print(Document doc) { /* ... */ }
    @Override public void scan(Document doc)  { /* ... */ }
}

public class OfficePrinter implements IPrintable, IScannable, IFaxable, IEmailable {
    @Override public void print(Document doc)  { /* ... */ }
    @Override public void scan(Document doc)   { /* ... */ }
    @Override public void fax(Document doc)    { /* ... */ }
    @Override public void email(Document doc)  { /* ... */ }
}
```

Real-World Application

- **Spring Data**: `CrudRepository`, `PagingAndSortingRepository`, `JpaRepository` — each layer adds methods; clients use only what they need
- **Repository pattern**: separate `ReadOrderRepository` and `WriteOrderRepository` — reporting services need read-only
- **REST API design**: different response DTOs for different clients (Backend-for-Frontend pattern)

Signs You Are Violating ISP

- Interface has 10+ methods with very different purposes
- Many implementations are mostly `throw new UnsupportedOperationException()`
- Mocking the interface for a unit test requires stubbing methods the test never calls

---

7. D — Dependency Inversion Principle

Core Idea

> A. High-level modules should **not** depend on low-level modules. Both should depend on **abstractions**. B. Abstractions should not depend on details. Details should depend on abstractions.

Mental Model

Think of a **power socket standard**. Your laptop (high-level) doesn't wire itself directly to a coal plant (low-level). Both depend on the socket standard (abstraction). You can swap the plant (coal → solar) without modifying the laptop.

Dependency Injection vs Dependency Inversion

Term	Meaning
Dependency Inversion	Design **principle** — depend on abstractions
Dependency Injection	Implementation **technique** — pass dependencies from outside
IoC Container	**Tool** that automates injection (Spring, Guice, Dagger)

DI is the most common way to achieve DIP, but DIP is the goal; DI is the mechanism.

Bad Example

```
// BAD — high-level OrderService depends on low-level MySQLOrderRepository
public class MySQLOrderRepository {
    public void save(Order order) {
        // JDBC logic tied to MySQL
    }
}

public class OrderService {
    private MySQLOrderRepository repository = new MySQLOrderRepository(); // tightly coupled

    public void placeOrder(Order order) {
        repository.save(order);
    }
}
```

Good Example

```
// GOOD — both depend on the IOrderRepository abstraction

public interface IOrderRepository {
    void save(Order order);
    Optional<Order> findById(long id);
}

public class MySQLOrderRepository implements IOrderRepository {
    @Override public void save(Order order)               { /* JDBC/MySQL logic */ }
    @Override public Optional<Order> findById(long id)    { /* JDBC/MySQL logic */ return
Optional.empty(); }
}

public class MongoOrderRepository implements IOrderRepository {
    @Override public void save(Order order)               { /* MongoDB logic */ }
    @Override public Optional<Order> findById(long id)    { /* MongoDB logic */ return
Optional.empty(); }
}

public class InMemoryOrderRepository implements IOrderRepository { // for unit tests
    private Map<Long, Order> store = new HashMap<>();
    @Override public void save(Order order)               { store.put(order.getId(), order); }
    @Override public Optional<Order> findById(long id)    { return
Optional.ofNullable(store.get(id)); }
}

// High-level module — depends only on the abstraction
public class OrderService {
    private final IOrderRepository repository;

    public OrderService(IOrderRepository repository) { // injected from outside
        this.repository = repository;
    }

    public void placeOrder(Order order) {
        repository.save(order);
    }
}

// Spring wiring (composition root)
@Configuration
public class AppConfig {
    @Bean
    public IOrderRepository orderRepository() { return new MySQLOrderRepository(); }

    @Bean
    public OrderService orderService(IOrderRepository repo) { return new OrderService(repo); }
}
```

Signs You Are Violating DIP

- `new ConcreteClass()` inside a `@Service` or `@Component` constructor
- Business logic imports `java.sql.*` directly
- Changing the DB layer (MySQL → Postgres) touches many service classes
- Unit tests require a running database

---

8. How the Principles Work Together

```
SRP  → small, focused classes with a clear purpose
OCP  → those classes are extended without modification
LSP  → extensions are safe substitutes; contracts are honoured
ISP  → clients depend only on what they use (smaller contracts)
DIP  → high-level code depends on those small contracts, not implementations
```

A typical design flow:

1. **SRP** — identify all responsibilities; split into separate classes
2. **ISP** — define small interfaces for each role those classes play
3. **DIP** — make high-level classes depend on those interfaces
4. **OCP** — add new behaviour by implementing the interface, not by if/else
5. **LSP** — verify every implementation fully honours the interface contract

---

9. SOLID vs Other Principles

Principle	Relationship with SOLID
DRY (Don't Repeat Yourself)	SRP centralises each concern, naturally reducing duplication
KISS (Keep It Simple)	ISP and SRP push toward simpler, smaller units
YAGNI (You Aren't Gonna Need It)	Do not add abstractions for OCP/DIP until variation is actually needed
Law of Demeter	Reinforces SRP and DIP — talk only to direct collaborators
Composition over Inheritance	Supports OCP and LSP — prefer delegation over deep hierarchies

---

10. Common Misconceptions

Misconception	Reality
SOLID means always creating an interface	Only when variation is expected or testability requires it
SRP means one method per class	One *reason to change*, not one method
OCP means never modifying code	Bug fixes are fine; hacking new behaviour into if/else is not
LSP only applies to inheritance	Also applies to interface implementations
DIP requires Spring or a DI framework	Manual constructor injection in a composition root is sufficient
SOLID is only for large systems	Small systems benefit too; the cost is low

---

Quick Cheat Sheet

```
S — One class, one reason to change
O — Extend by adding new classes, not by editing existing ones
L — Subclass must keep every promise the parent made
I — Small focused interfaces beat one fat interface
D — Depend on what something does (interface), not what it is (class)
```

Part 2: Interview Questions and Answers

All questions — Basic → Intermediate → Advanced → Real-World Use Cases → Coding Challenges

---

SECTION 1 — BASIC

Q1. What is SOLID, and why do we use it?

SOLID is a set of five design principles for object-oriented programming coined by Robert C. Martin.

We use it because without these principles, large codebases become:

- **Rigid** — one small change breaks many unrelated things
- **Fragile** — modules break in unexpected places
- **Immobile** — hard to reuse code in new contexts

With SOLID, code is easier to **test**, **extend**, and **maintain**.

Q2. What are the five SOLID principles?

Letter	Principle	Core Idea
S	Single Responsibility	One class = one reason to change
O	Open/Closed	Open for extension, closed for modification
L	Liskov Substitution	Subtypes must be substitutable for their base types
I	Interface Segregation	Many small interfaces beat one fat interface
D	Dependency Inversion	Depend on abstractions, not concrete implementations

---

Q3. What problem does SOLID solve in OOP?

Without SOLID, systems develop four symptoms:

1. **Rigidity** — changing one thing forces changes everywhere
2. **Fragility** — unrelated modules break from a single change
3. **Immobility** — can't extract and reuse code safely
4. **Viscosity** — hacking is faster than doing it right

SOLID makes code **tolerant of change**, **easy to understand**, and **reusable**.

Q4. Explain SRP with a Java example.

A class should have **one, and only one, reason to change**.

```java
// BAD — three different stakeholders can force changes
public class UserService {
    public User register(UserDto dto)      { /* business logic */ }
    public void sendWelcomeEmail(User u)   { /* email logic */ }
    public void save(User user)            { /* DB logic */ }
}

// GOOD — each class has one reason to change
public class UserRegistrationService { public User register(UserDto dto) { /* ... */ } }
public class EmailService            { public void sendWelcome(User u)   { /* ... */ } }
public class UserRepository          { public void save(User user)        { /* ... */ } }
```

Q5. Does SRP mean a class should have only one method?

No. SRP is about one **reason to change** (one stakeholder), not one method.

```java
// Multiple methods — still ONE responsibility (pricing rules)
public class OrderPricingCalculator {
    public double calculateSubtotal(Order order) { /* ... */ }
    public double calculateTax(Order order)       { /* ... */ }
    public double calculateShipping(Order order)  { /* ... */ }
    public double calculateTotal(Order order)     { /* ... */ }
}
```

Q6. What does 'one reason to change' mean in SRP?

A "reason to change" maps to a **stakeholder** or **actor** — a person or group whose requirements drive changes.

Ask: "*If I change this class, who benefits?*" If the answer is Finance team AND HR team AND DBA for different things, that class has three reasons to change — split it.

Q7. Explain OCP in simple words.

- **Open for extension** = add new behaviour by adding new code
- **Closed for modification** = don't touch working, tested code

```java
// Closed — never changes
public class AreaCalculator {
    public double calculate(Shape shape) { return shape.area(); }
}

// Open — extend by adding new Shape implementations
public class Pentagon implements Shape {
    public double area() { /* pentagon formula */ return 0; }
}
```

Q8. What does 'open for extension, closed for modification' mean?

Adding new behaviour means adding a new class that implements an existing interface. The calling code is never edited. Polymorphism is the mechanism.

Analogy: a USB port — plug in new devices (extension) without rewiring the motherboard (modification).

Q9. What is LSP?

If `S` is a subtype of `T`, you must be able to replace any `T` with an `S` **without the program breaking**.

```
// LSP violation — Square breaks Rectangle's contract
class Rectangle {
    void setWidth(int w)  { this.width = w; }
    void setHeight(int h) { this.height = h; }
    int area()            { return width * height; }
}
class Square extends Rectangle {
    @Override void setWidth(int w) { this.width = this.height = w; } // side-effect
}

void test(Rectangle r) {
    r.setWidth(4); r.setHeight(5);
    assert r.area() == 20; // FAILS for Square — area is 25
}
```

Q10. What does substitutability mean in Java?

Code written for a base type (interface or parent class) works correctly with any implementation or subclass. No surprises, no extra type checks, no unexpected exceptions.

Q11. What is ISP?

No client should be forced to implement or depend on methods it does not use. Split large interfaces into smaller, focused ones.

```
// BAD
public interface AllInOneMachine { void print(); void scan(); void fax(); void email(); }

// GOOD
public interface Printable { void print(); }
public interface Scannable { void scan(); }

public class BasicPrinter implements Printable { public void print() { /* ... */ } }
```

Q12. Why are small, focused interfaces useful?

1. Classes implement only what they actually support
2. Mocks have fewer methods to stub in tests
3. Interface names describe a clear role (`Printable`, `Auditable`)
4. Clients depend only on what they use — loose coupling

5. Adding a new role = new interface, not expanding an existing one

---

Q13. What is DIP?

1. High-level modules should not depend on low-level modules. Both should depend on **abstractions**.
2. Abstractions should not depend on details. Details should depend on abstractions.

```
// BAD
public class OrderService {
    private MySQLDatabase db = new MySQLDatabase(); // tightly coupled
}

// GOOD
public interface IDatabase { void save(Order o); }
public class OrderService {
    private final IDatabase db;
    public OrderService(IDatabase db) { this.db = db; } // injected
}
```

Q14. What is the difference between DIP and Dependency Injection?

	DIP	DI
What	Design **principle**	Implementation **technique**
Goal	Depend on abstractions	Pass dependencies from outside

DI is the most common way to satisfy DIP. You can have DI without DIP (injecting a concrete class).

Q15. How does SOLID improve maintainability and testability?

Principle	Maintainability	Testability
SRP	Small classes are easy to change	Each unit tests one thing
OCP	Adding features doesn't risk breaking old code	Old tests stay valid
LSP	Polymorphism is reliable — no subclass surprises	Test doubles are safe
ISP	Fewer, relevant dependencies	Smaller mocks
DIP	Swap implementations without touching business logic	Inject test doubles freely

---

Q16. Give a real-world analogy for each principle.

Principle	Analogy
SRP	Chef cooks, waiter serves — one person doesn't do both
OCP	USB port — plug in new devices without rewiring the motherboard
LSP	Subcontractor follows the same building codes as the main contractor
ISP	TV remote vs universal remote — expose only relevant buttons
DIP	Power socket standard — laptop plugs into any socket, not a specific plant

---

SECTION 2 — INTERMEDIATE

SRP — Intermediate

Q17. Can a class with 500 lines follow SRP?

Yes, if every line serves the same single responsibility. Line count is not the measure.
In practice, 500-line classes usually contain multiple responsibilities. The test: how many different stakeholders could ask you to change it?

Q18. How do you identify multiple responsibilities in a service class?

- Imports from different domains: `java.sql.*` AND `javax.mail.*` AND `com.itextpdf.*`
- Methods with unrelated concerns: `validateUser()`, `sendEmail()`, `generateReport()` in one class
- Different change triggers: Finance, DBA, and Email team can each independently demand changes
- Test difficulty: testing one method requires mocking 5 unrelated collaborators

---

Q19. Should validation, persistence, and email delivery belong in one service?

No. These are three separate responsibilities owned by different teams.

```
// GOOD — each concern is a separate collaborator
public class OrderService {
    private final OrderValidator validator;
    private final OrderRepository repository;
    private final OrderNotifier notifier;

    public void placeOrder(OrderDto dto) {
        validator.validate(dto);
        Order order = repository.save(toEntity(dto));
        notifier.notifyCustomer(order);
    }
}
```

Q20. How do you split a class without creating too many small classes?

- Split along **actor boundaries**, not line count
- Group methods that always change together in the same class
- Use package structure to keep related classes discoverable
- Don't create one-method classes unless the method has complex dependencies of its own

---

Q21. What is the difference between responsibility and functionality?

	Responsibility	Functionality
Definition	A reason to change — who owns it	What the code does

One *functionality* (place an order) can involve multiple responsibilities (validate, persist, notify). SRP is about responsibilities, not features.

Q22. Can a controller violate SRP even with only a few methods?

Yes — if it contains business logic, DB queries, or email sending.

```
// BAD — business logic in controller
@RestController
public class OrderController {
    @PostMapping("/orders")
    public ResponseEntity<?> placeOrder(@RequestBody OrderDto dto) {
        if (dto.getTotal() > 10000) applySpecialDiscount(dto); // business logic
        orderRepository.save(toEntity(dto));                    // persistence
        return ResponseEntity.ok().build();
    }
}

// GOOD — controller only handles HTTP concerns
@RestController
public class OrderController {
    @PostMapping("/orders")
    public ResponseEntity<?> placeOrder(@RequestBody OrderDto dto) {
        orderService.placeOrder(dto);
        return ResponseEntity.ok().build();
    }
}
```

Q23. How do business stakeholders help identify reasons to change?

Ask: "If ___ team changes requirements, which class would I edit?"

- Finance team → `PayCalculator`
- DBA → `UserRepository`
- Comms team → `EmailNotificationService`

If two different teams point to the same class, split it.

Q24. How would you refactor a UserService that handles registration, password reset, email sending, and DB access?

```java
// Four focused classes instead of one god class
public class UserRegistrationService {
    private final UserRepository repo;
    private final EmailService email;
    public void register(UserDto dto) { repo.save(toEntity(dto));
email.sendWelcome(dto.getEmail()); }
}

public class PasswordResetService {
    private final UserRepository repo;
    private final TokenService tokens;
    private final EmailService email;
    public void reset(String email) { /* token + save + email */ }
}

public class EmailService {
    public void sendWelcome(String address)                        { /* SMTP */ }
    public void sendPasswordReset(String address, String token)    { /* SMTP */ }
}

public class UserRepository {
    public void save(User user)                          { /* JDBC */ }
    public Optional<User> findByEmail(String email)      { return Optional.empty(); }
}
```

OCP — Intermediate

Q25. How do you replace a long if-else chain with polymorphism?

Use the Strategy Pattern: each branch becomes a class implementing a common interface.

```java
public interface NotificationStrategy {
    void send(String message);
    boolean supports(String type);
}

@Component public class EmailStrategy implements NotificationStrategy {
    public void send(String msg) { /* SMTP */ }
    public boolean supports(String t) { return "EMAIL".equals(t); }
}

@Service
public class NotificationService {
    private final List<NotificationStrategy> strategies;
    @Autowired public NotificationService(List<NotificationStrategy> strategies) {
this.strategies = strategies; }

    public void send(String type, String message) {
        strategies.stream().filter(s -> s.supports(type)).findFirst()
            .orElseThrow(() -> new IllegalArgumentException("No strategy for type: " + type))
            .send(message);
    }
}
// Adding SLACK = new @Component class — NotificationService never changes
```

Q26. How can you add a new payment type without changing existing payment logic?

```java
public interface PaymentStrategy { PaymentResult pay(PaymentRequest req); String getType(); }

@Component public class CreditCardPayment implements PaymentStrategy { /* Stripe */ public
String getType() { return "CREDIT_CARD"; } }
@Component public class CryptoPayment    implements PaymentStrategy { /* blockchain */ public
String getType() { return "CRYPTO"; } }

@Service
public class PaymentProcessor {
    private final Map<String, PaymentStrategy> strategies;
    @Autowired
    public PaymentProcessor(List<PaymentStrategy> list) {
        this.strategies = list.stream().collect(Collectors.toMap(PaymentStrategy::getType, s -
> s));
    }
    public PaymentResult process(String type, PaymentRequest req) {
        PaymentStrategy strategy = strategies.get(type);
        if (strategy == null) throw new IllegalArgumentException("Unknown payment type: " +
type);
        return strategy.pay(req);
    }
}
```

Q27. Does OCP mean you should never modify existing code?

No. Bug fixes and refactoring are fine. The violation is when **adding a new feature** requires editing a class unrelated to that feature.

Q28. How do interfaces and abstract classes help implement OCP?

Tool	Role
Interface	Stable contract; new behaviour = new implementation class
Abstract class	Fixed skeleton (Template Method); new behaviour = override variable steps

---

Q29. Can Strategy and Factory patterns help implement OCP?

Yes.

- **Strategy**: new variant = new class implementing the strategy interface. Context never changes.
- **Factory**: new types register with the factory. Callers who use the factory never change.

Q30. When can OCP lead to unnecessary abstractions?

When you create interfaces and extension points for things that will never actually vary. This is YAGNI violation. Apply OCP at the **second** implementation, not speculatively at the first.

Q31. How would you design a discount engine that supports new discount rules?

```java
public interface DiscountRule {
    boolean isApplicable(Order order);
    double apply(double total);
}

@Component public class LoyaltyDiscount implements DiscountRule {
    public boolean isApplicable(Order o) { return o.getCustomer().isLoyaltyMember(); }
    public double apply(double total)     { return total * 0.85; }
}

@Service
public class DiscountEngine {
    private final List<DiscountRule> rules;
    @Autowired public DiscountEngine(List<DiscountRule> rules) { this.rules = rules; }

    public double applyBestDiscount(Order order) {
        return rules.stream().filter(r -> r.isApplicable(order))
            .mapToDouble(r -> r.apply(order.getTotal())).min().orElse(order.getTotal());
    }
}
// New discount type = new @Component class. DiscountEngine never changes.
```

LSP — Intermediate

Q32. Give an example of inheritance that violates LSP.

```java
// ReadOnlyList throws on mutating operations — violates LSP
public class ReadOnlyList<T> extends ArrayList<T> {
    @Override
    public boolean add(T element) { throw new UnsupportedOperationException(); }
}
// Code written for List<T> breaks at runtime when handed a ReadOnlyList
```

Q33. Why is the Rectangle/Square example the classic LSP illustration?

Because it feels mathematically correct (a square IS a rectangle), yet the code contract breaks. `Rectangle` guarantees independent width/height. `Square` couples them — any caller that sets them independently and checks `area()` will get a wrong result.

It teaches: **mathematical "is-a" ≠ behavioural substitutability**.

Q34. Can a child class strengthen preconditions or weaken postconditions?

No — both violate LSP.

- **Strengthening preconditions**: parent accepts `null`, child throws `NullPointerException` — callers break
- **Weakening postconditions**: parent guarantees non-null return, child may return `null` — callers break

---

Q35. What happens when a subclass throws UnsupportedOperationException?

It is an LSP violation. The parent's contract says the operation works. Callers relying on that contract will get a runtime exception they cannot anticipate.

Fix: use composition instead of inheritance, or extract a common interface that only declares what the subclass actually supports.

Q36. How do you preserve behavioural contracts when overriding a method?

1. Accept at least the same inputs (don't add extra checks)
2. Return at least the same guarantees (don't weaken return type or nullability)
3. Preserve class-level invariants
4. Don't throw new checked exceptions
5. Run the parent's test suite against the subclass

Q37. Can composition fix an LSP violation better than inheritance?

Yes. When the subclass can't honour the parent's contract, don't inherit — compose.

```
// SavingsAccount can't honour BasicAccount's withdraw contract fully
// → wrap instead of extend
public class SavingsAccount {
    private final BasicAccount account; // composed
    public void withdraw(double amount) {
        if (account.getBalance() - amount < minimumBalance) throw new
InsufficientFundsException();
        account.withdraw(amount);
    }
}
```

Q38. How would you model read-only and writable data sources without violating LSP?

```
public interface ReadableRepository<T, ID> {
    Optional<T> findById(ID id);
    List<T> findAll();
}

public interface WritableRepository<T, ID> extends ReadableRepository<T, ID> {
    void save(T entity);
    void delete(ID id);
}

public class CacheRepository    implements ReadableRepository<Product, Long> { /* read-only */
}
public class JpaProductRepository implements WritableRepository<Product, Long> { /* full CRUD
*/ }
```

Q39. How can unit tests help verify LSP (substitutability)?

Write a **contract test** (abstract test class) and run it against every implementation.

```
public abstract class NotificationServiceContract {
    protected abstract INotificationService createService();

    @Test void send_shouldNotThrowForValidMessage() {
        assertDoesNotThrow(() -> createService().send("Hello", "user@example.com"));
    }
    @Test void send_shouldThrowForNullMessage() {
        assertThrows(IllegalArgumentException.class,
            () -> createService().send(null, "user@example.com"));
    }
}

class EmailServiceTest extends NotificationServiceContract {
    protected INotificationService createService() { return new
EmailNotificationService(mockSmtp()); }
}
class SMSServiceTest extends NotificationServiceContract {
    protected INotificationService createService() { return new
SMSNotificationService(mockTwilio()); }
}
```

ISP — Intermediate

Q40. What is a fat interface?

An interface that declares more methods than any single client needs, forcing implementations to stub methods they don't support. Signs: 10+ unrelated methods, many `throw new UnsupportedOperationException()`, painful mocking.

Q41. How do you identify an ISP violation?

- `throw new UnsupportedOperationException()` in an implementation
- A class implements an interface but only uses 1–2 of its 10 methods
- Adding a method forces changes in many unrelated classes
- Javadoc says "optional operation" for some methods

Q42. Should every implementation implement every method in an interface?

Yes — that is the contract. If a class can't meaningfully implement a method, it shouldn't implement that interface. ISP exists to make this always possible by keeping interfaces small.

Q43. Why is an interface with `default` methods not automatically ISP-compliant?

`default` methods hide the violation syntactically. A `Robot` implementing a `Worker` interface with a `default eat()` method still carries an irrelevant dependency. The correct fix is still splitting the interface.

Q44. How would you split a large Worker interface into focused interfaces?

```java
// BAD
public interface Worker { void work(); void eat(); void sleep(); void submitTimesheet(); }

// GOOD
public interface Workable   { void work(); }
public interface Feedable   { void eat(); }
public interface Reportable { void submitTimesheet(); }

public class Human implements Workable, Feedable, Reportable { /* all */ }
public class Robot implements Workable { /* only work */ }
```

Q45. Can too many small interfaces make a system harder to understand?

Yes. Over-applying ISP produces dozens of single-method interfaces that fragment a coherent concept. Balance: group methods that are **always used together**. Split only when some clients genuinely never need some methods.

Q46. How would you design separate interfaces for reading and writing data?

```java
public interface OrderReader { Optional<Order> findById(long id); List<Order> findAll(); }
public interface OrderWriter { Order save(Order o); void delete(long id); }
public interface OrderRepository extends OrderReader, OrderWriter {} // full CRUD

// Reporting service — read-only dependency
@Service
public class OrderReportService {
    private final OrderReader reader; // doesn't need write capability
    public OrderReportService(OrderReader reader) { this.reader = reader; }
}
```

Q47. How does ISP help when multiple teams work on the same codebase?

- Team A adding a method to their write interface doesn't affect Team B using only the read interface
- Each team mocks only the methods they care about — less inter-team coupling in tests
- Interface boundaries become clear team ownership lines (Conway's Law)

DIP — Intermediate

Q48. What are high-level and low-level modules?

	Definition	Example
High-level	Business logic, use cases	`OrderService`, `PaymentProcessor`
Low-level	I/O, DB, network details	`MySQLOrderRepository`, `SmtpEmailService`

DIP says both should depend on an abstraction (interface), not on each other.

Q49. Why is `new ConcreteRepository()` inside a service problematic?

1. Untestable — needs a real DB to test the service

2. Tightly coupled — switching DB means modifying the service
3. Violates DIP — high-level depends on low-level
4. Hides dependencies — callers can't see what the service needs

---

Q50. How do constructor injection and interfaces support DIP?

Constructor injection makes dependencies **explicit and swappable**. The interface makes them **abstract**.

```
public class OrderService {
    private final IOrderRepository repo;
    private final IEmailService email;
    public OrderService(IOrderRepository repo, IEmailService email) {
        this.repo = repo; this.email = email;
    }
}
// Test: inject InMemoryOrderRepository + MockEmailService
// Prod: inject JpaOrderRepository + SmtpEmailService
```

Q51. Does `@Autowired` automatically mean the design follows DIP?

No. DIP requires the injected type to be an **interface**. Injecting a concrete class with `@Autowired` is DI without DIP.

Q52. Should domain code depend on Spring annotations?

In strict Clean Architecture: no. Domain should be framework-agnostic for portability and testability. Many Spring Boot projects accept the trade-off pragmatically, but the coupling should be a conscious choice.

Q53. Where should an abstraction (interface) be defined — business layer or infrastructure layer?

In the **business/domain layer**. The domain owns the contract; infrastructure implements it.

```
domain/ports/IOrderRepository.java          ← abstraction (domain)
infrastructure/adapters/JpaOrderRepository.java  ← implementation (infra)
```

If the interface lives in infrastructure, the domain still indirectly depends on infrastructure.

Q54. What are the trade-offs of introducing interfaces around every dependency?

Benefit	Cost
Easy to swap implementations	Extra file per dependency
Easy to mock in tests	Indirection makes code harder to trace
Loose coupling	Over-abstraction for stable, never-varying classes

Guideline: introduce an interface when the class touches I/O, needs to be mocked, or will have multiple implementations.

SECTION 3 — ADVANCED

Q55. How do the five SOLID principles reinforce one another?

```
SRP  → small, focused classes with a clear purpose
OCP  → those classes are extended without modification
LSP  → extensions are safe substitutes; contracts are honoured
ISP  → clients depend only on what they use (smaller contracts)
DIP  → high-level code depends on those small contracts, not implementations
```

Typical design flow:

1. SRP — split responsibilities into separate classes
2. ISP — define small interfaces for each role
3. DIP — high-level code depends on those interfaces
4. OCP — new behaviour = new implementation of the interface
5. LSP — every implementation fully honours the interface contract

---

Q56. Can following one SOLID principle appear to conflict with another?

Yes. Common tensions:

Tension	Resolution
SRP vs DRY	Extract shared logic to a third class; don't merge the two classes
OCP vs YAGNI	Apply OCP when variation is already known, not speculatively
ISP vs Cohesion	Group methods that are always used together; split only when clients diverge
DIP vs Simplicity	Skip the interface when there's one implementation and no test mock needed

When in doubt: ask "*what is the cost of the wrong choice?*" Missing an abstraction costs more later than adding one too early in most enterprise systems.

Q57. How would you apply SOLID in a legacy application with limited refactoring time?

Use the **Strangler Fig Pattern**:

1. **Write characterisation tests first** — understand current behaviour before changing anything
2. **Extract interfaces** in front of concrete classes you want to replace
3. **Delegate from the old class** to new focused classes — existing callers still work
4. **Migrate callers** one by one to use the new classes directly
5. **Delete the old class** once it has no callers

```java
// Step 1: Legacy god class
public class OrderManager {
    public void placeOrder(Order order)      { /* 200 lines */ }
    public void sendEmail(Order order)       { /* 80 lines */ }
    public void updateInventory(Order order) { /* 60 lines */ }
}

// Step 2: Extract interfaces + focused classes (one at a time)
public interface IOrderPlacer   { void placeOrder(Order order); }
public class OrderPlacer implements IOrderPlacer { /* extracted logic */ }

// Step 3: Legacy class delegates (old callers still work)
public class OrderManager {
    private final OrderPlacer placer = new OrderPlacer(...);
    public void placeOrder(Order order) { placer.placeOrder(order); }
}
```

Q58. How do you measure whether an abstraction is justified?

An abstraction is justified when at least one is true:

1. There are (or will soon be) **multiple implementations**
2. The class touches **I/O** and needs to be replaced in tests
3. A **change in the low-level module** should not affect the high-level module

Skip the abstraction when: one implementation, never mocked, stable forever (pure computation). Introducing it adds indirection with no payoff.

Q59. How do SOLID principles relate to Clean Architecture and Hexagonal Architecture?

Both architectures are built directly on SOLID:

SOLID	Clean / Hexagonal role
SRP	Each layer has one reason to change: Entities (business rules), Use Cases (app logic), Adapters (delivery)
OCP	Inner layers never change when outer layers (DB, framework) change
LSP	Repository interface in the domain; MySQL/Mongo are substitutable implementations
ISP	Use Cases expose narrow ports (e.g., `IUserReader`, not full `IUserRepository`)
DIP	Domain defines interfaces (ports); Adapters implement them — all dependencies point inward

The **Dependency Rule**: source code dependencies must always point inward (toward business rules).

Q60. How does DIP enforce the dependency rule in Clean Architecture?

```
domain/ports/IOrderRepository.java    ← interface defined in domain
infra/JpaOrderRepository.java         ← implementation in infrastructure, depends on domain

// At runtime: Spring injects JpaOrderRepository into domain's OrderService
// At compile time: domain has zero import of infrastructure packages
```

`OrderService` (domain) imports `IOrderRepository` (domain). `JpaOrderRepository` (infra) imports `IOrderRepository` (domain). The infra layer depends on the domain — never the reverse.

Q61. How do SOLID principles apply to microservices architecture?

Principle	Microservices equivalent
SRP	Each service owns one bounded context
OCP	New features are new services or service versions, not edits to live services
LSP	Services must honour their API contract — v2 must not break v1 callers
ISP	Expose only the endpoints each consumer needs; use BFF (Backend-for-Frontend)
DIP	Services communicate via API contracts (OpenAPI/Protobuf), not internal details

Event-driven DIP example:

```
OrderService → publishes OrderPlaced event → Kafka
InventoryService → subscribes, decrements stock
EmailService → subscribes, sends confirmation
// Both depend on the event contract (abstraction) — not on each other
```

Q62. How does LSP apply to exception handling in Java?

A subtype must not throw new **checked exceptions** not declared by the base method signature.

```java
// LSP violation — throws undeclared checked exception
public class S3FileReader implements FileReader {
    public String read(String path) throws IOException, AmazonS3Exception { /* */ }
}

// LSP-compliant — translate to the declared exception
public class S3FileReader implements FileReader {
    public String read(String path) throws IOException {
        try { return s3Client.getObject(bucket, path); }
        catch (AmazonS3Exception e) { throw new IOException("S3 read failed", e); }
    }
}
```

Q63. How do you handle the 'explosion of interfaces' when aggressively applying ISP + DIP?

1. Name interfaces after the **role** they represent (`Authenticatable`, `Auditable`), not the class (`IUserImpl`)
2. Package by feature — keep interface and implementation together
3. Accept that not everything needs an interface (pure computation, no I/O, never mocked)
4. Use Java 8 `default` methods to evolve interfaces without breaking all implementors
5. A class that will **never** have a second implementation and will **never** be mocked doesn't need an interface

---

Q64. How do SOLID principles relate to DDD (Domain-Driven Design)?

DDD concept	SOLID connection
Bounded Context	SRP at module level
Repository	DIP — domain defines the port; infrastructure provides the adapter
Domain Event	OCP — new reactions = new subscribers, not changes to the publisher
Anti-Corruption Layer	DIP — isolates domain from external models
Aggregate	SRP — one aggregate root per consistency boundary

Q65. What are the risks of using SOLID as a rigid checklist?

- Over-engineering: interfaces everywhere, even where one implementation will ever exist
- Abstraction for its own sake: makes code harder to read and trace
- Premature design: solving problems that don't yet exist
- Slowing down early-stage development where flexibility is still needed

SOLID is a **set of guidelines** to help you reason about trade-offs, not a law to obey blindly.

Q66. How would you justify a SOLID-related design decision in a code review?

Frame it as a concrete problem, not a principle violation:

- "If we add a new payment type, we'll need to edit this class. Here's how to prevent that."
- "To test `OrderService` in isolation, I need to inject a fake repository. Currently that's impossible."
- "When Finance changes the pay calculation, we risk breaking the email formatting logic in the same class."

Make the **cost of not applying the principle** visible and specific.

SECTION 4 — REAL-WORLD SPRING BOOT SCENARIOS

Scenario 1 — SRP: OrderService that does too much

Problem: `OrderService` validates orders, saves them, sends confirmation emails, and generates PDF invoices. A change in PDF format breaks order validation tests.

Violation: Multiple responsibilities — validation, persistence, communication, document generation.

Fix:

```
@Service public class OrderValidationService { public void validate(OrderDto dto) { /* ... */
} }
@Repository public class OrderRepository    { public Order save(Order o)            { return
null; } }
@Service public class OrderNotifier        { public void confirm(Order o)          { /* email
*/ } }
@Service public class InvoiceGenerator     { public byte[] generate(Order o)       { return
null; } }

@Service
public class OrderService {
    public Order placeOrder(OrderDto dto) {
        validator.validate(dto);
        Order order = repository.save(toEntity(dto));
        notifier.confirm(order);
        return order;
    }
}
```

Scenario 2 — OCP: Payment switch statement

Problem: Adding PayPal requires editing a 200-line `processPayment(String type, ...)` switch block — risking regressions in working Stripe and Bank Transfer code.

Fix: Strategy Pattern (see Q25 above). Each payment type is a `@Component` implementing `PaymentStrategy`. `PaymentProcessor` never changes.

Scenario 3 — LSP: ReadOnlyRepository that fails on save()

Problem: `ReadOnlyProductRepository extends JpaRepository`. A reporting service calls it, but a developer accidentally calls `.save()` — it succeeds (no compile error), corrupts data. Later, if `save()` is made to throw, LSP is violated.

Fix:

```
// Don't extend JpaRepository — define your own read-only interface
public interface ReadOnlyProductRepository extends Repository<Product, Long> {
    Optional<Product> findById(Long id);
    List<Product> findByCategory(String category);
}
// No save(), delete(), etc. — impossible to misuse
```

Scenario 4 — ISP: ReportExporter with irrelevant methods

Problem: Every report exporter must implement `sendEmail()` and `uploadToCloud()` even if it only writes to a local file.

Fix:

```
public interface ReportExporter       { byte[] export(Report report); }
public interface EmailDeliverable     { void sendEmail(byte[] report, String recipient); }
public interface CloudUploadable      { String uploadToCloud(byte[] report); }

public class PDFExporter    implements ReportExporter { /* generates PDF */ }
public class FullExporter   implements ReportExporter, EmailDeliverable, CloudUploadable { /*
all */ }
```

Scenario 5 — DIP: Service that creates its own SQL repository

Problem: `InvoiceService` contains `private InvoiceSQLRepository repo = new InvoiceSQLRepository()`. Unit tests require a real DB. Switching to MongoDB means editing `InvoiceService`.

Fix:

```
public interface InvoiceRepository { void save(Invoice invoice); Optional<Invoice>
findById(long id); }

@Service
public class InvoiceService {
    private final InvoiceRepository repo;
    @Autowired public InvoiceService(InvoiceRepository repo) { this.repo = repo; }
}
// Test: inject InMemoryInvoiceRepository
// Prod: Spring autowires JpaInvoiceRepository
```

Scenario 6 — OCP + DIP: Replacing an SMS provider breaks notification code

Problem: `NotificationService` has `private TwilioClient twilio = new TwilioClient()`. Replacing Twilio with AWS SNS requires editing `NotificationService` and re-testing all notification logic.

Fix: Extract `ISmsProvider` interface; inject. Adding AWS SNS = new class, no edits to `NotificationService`.

Scenario 7 — LSP: Subclass changes return value meaning

Problem: Base `PriceCalculator.calculate()` returns the final price. A subclass `VIPPriceCalculator` returns the **discount amount** instead. Callers using the base type display wrong prices.

Fix: Override must return the same *meaning*: the final price. If the semantics are different, it should be a separate method, not an override.

Scenario 8 — DIP: Service needs real DB for unit test

Problem: `ProductService` creates `new JdbcProductRepository()` internally. Every unit test spins up H2 or Docker — slow, flaky CI.

Fix: Inject `IProductRepository`. Test injects `InMemoryProductRepository`. Unit test runs in milliseconds with no DB.

Scenario 9 — SRP: One change to invoice formatting breaks payment processing

Problem: `BillingService` contains both `formatInvoice()` and `chargeCard()`. A layout change to the invoice format introduces a null pointer that crashes the card charging logic.

Fix: `InvoiceFormatter` (owned by Finance/UX) and `PaymentProcessor` (owned by Payments team) are separate classes that change for separate reasons.

Scenario 10 — ISP: Service depends on large interface with unrelated operations

Problem: `ReportingService` depends on `IOrderService` which has 30 methods — placement, cancellation, tracking, returns. Reporting only needs `findAll()` and `findByDateRange()`.

Fix:

```
public interface OrderQueryService {
    List<Order> findAll();
    List<Order> findByDateRange(LocalDate from, LocalDate to);
}

@Service
public class ReportingService {
    private final OrderQueryService queryService; // depends only on what it uses
}
```

SECTION 5 — CODING CHALLENGES

Challenge A — SRP: Refactor InvoiceService

Problem: `InvoiceService` calculates totals, saves invoices, sends emails, and generates PDFs. Refactor into focused components.

Solution:

```java
// Focused classes
public class InvoiceTotalCalculator {
    public double calculate(Invoice invoice) {
        return invoice.getItems().stream()
            .mapToDouble(i -> i.getQuantity() * i.getUnitPrice()).sum();
    }
}

public class InvoiceRepository {
    public Invoice save(Invoice invoice) { /* JPA */ return invoice; }
    public Optional<Invoice> findById(long id) { return Optional.empty(); }
}

public class InvoiceEmailSender {
    public void send(Invoice invoice, String customerEmail) { /* SMTP */ }
}

public class InvoicePdfGenerator {
    public byte[] generate(Invoice invoice) { /* iText / Apache PDFBox */ return new byte[0];
}
}

// Orchestrator
@Service
public class InvoiceService {
    private final InvoiceTotalCalculator calculator;
    private final InvoiceRepository repository;
    private final InvoiceEmailSender emailSender;
    private final InvoicePdfGenerator pdfGenerator;

    public InvoiceService(InvoiceTotalCalculator calculator,
                          InvoiceRepository repository,
                          InvoiceEmailSender emailSender,
                          InvoicePdfGenerator pdfGenerator) {
        this.calculator   = calculator;
        this.repository   = repository;
        this.emailSender  = emailSender;
        this.pdfGenerator = pdfGenerator;
    }

    public Invoice createAndDeliver(InvoiceDto dto, String customerEmail) {
        Invoice invoice = toEntity(dto);
        invoice.setTotal(calculator.calculate(invoice));
        Invoice saved = repository.save(invoice);
        emailSender.send(saved, customerEmail);
        return saved;
    }

    private Invoice toEntity(InvoiceDto dto) {
        Invoice invoice = new Invoice();
        invoice.setCustomerId(dto.getCustomerId());
        invoice.setItems(dto.getItems());
        return invoice;
    }
}
```

Challenge B — OCP: Discount Calculator

Problem: Support percentage, fixed, and future discount types without repeatedly editing a conditional block.

Solution:

```java
public interface DiscountStrategy {
    double apply(double originalPrice);
}

public class PercentageDiscount implements DiscountStrategy {
    private final double percent;
    public PercentageDiscount(double percent) { this.percent = percent; }
    public double apply(double price)          { return price * (1 - percent / 100); }
}

public class FixedDiscount implements DiscountStrategy {
    private final double amount;
    public FixedDiscount(double amount) { this.amount = amount; }
    public double apply(double price)    { return Math.max(0, price - amount); }
}

public class BuyOneGetOneFreeDiscount implements DiscountStrategy { // new — no existing code
changed
    public double apply(double price) { return price / 2; }
}

// OCP payoff: callers apply any strategy directly — no conditional logic anywhere
// Tests
@Test
void percentageDiscountApplied() {
    assertEquals(90.0, new PercentageDiscount(10).apply(100.0));
}
@Test
void fixedDiscountApplied() {
    assertEquals(85.0, new FixedDiscount(15).apply(100.0));
}
@Test
void bogoDiscountApplied() {
    assertEquals(50.0, new BuyOneGetOneFreeDiscount().apply(100.0));
}
```

Challenge C — LSP: Ostrich extends Bird

Problem: `Ostrich extends Bird`, but `Bird` defines `fly()`. Callers that call `bird.fly()` break when handed an `Ostrich`.

Solution:

```java
// Separate the capability that not all birds share
public interface Bird {
    void eat();
    void makeSound();
}

public interface FlyingBird extends Bird {
    void fly();
}

public class Eagle implements FlyingBird {
    public void eat()       { System.out.println("Eagle eating"); }
    public void makeSound() { System.out.println("Screech"); }
    public void fly()       { System.out.println("Eagle soaring"); }
}

public class Ostrich implements Bird {
    public void eat()       { System.out.println("Ostrich eating"); }
    public void makeSound() { System.out.println("Boom"); }
    // no fly() — LSP satisfied, no UnsupportedOperationException
}

// Caller only asks FlyingBird to fly — safe substitution
public void makeFly(FlyingBird bird) { bird.fly(); }
```

Challenge D — ISP: Split Machine Interface

Problem:

```java
interface Machine { void print(); void scan(); void fax(); }
```

Some devices can only print; others can print and scan. None should be forced to stub `fax()`.

Solution:

```java
public interface Printable { void print(); }
public interface Scannable { void scan(); }
public interface Faxable   { void fax(); }

public class BasicPrinter implements Printable {
    public void print() { System.out.println("Printing..."); }
}

public class OfficeScanner implements Printable, Scannable {
    public void print() { System.out.println("Printing..."); }
    public void scan()  { System.out.println("Scanning..."); }
}

public class AllInOnePrinter implements Printable, Scannable, Faxable {
    public void print() { System.out.println("Printing..."); }
    public void scan()  { System.out.println("Scanning..."); }
    public void fax()   { System.out.println("Faxing..."); }
}
// No class has an unused method — no UnsupportedOperationException needed
```

Challenge E — DIP: Refactor NotificationService

Problem: `NotificationService` directly creates `new EmailClient()`. Email can't be swapped for SMS or mocked in tests.

Solution:

```
// 1. Define the abstraction
public interface INotificationSender {
    void send(String recipient, String message);
}

// 2. Concrete implementations
public class EmailSender implements INotificationSender {
    private final EmailClient client;
    public EmailSender(EmailClient client) { this.client = client; }
    public void send(String recipient, String message) { client.send(recipient, message); }
}

public class SmsSender implements INotificationSender {
    private final SmsGateway gateway;
    public SmsSender(SmsGateway gateway) { this.gateway = gateway; }
    public void send(String recipient, String message) { gateway.sendText(recipient, message);
}
}

// 3. High-level service depends only on abstractions
@Service
public class NotificationService {
    private final INotificationSender sender;
    private final IUserContactRepository userRepository;

    @Autowired
    public NotificationService(INotificationSender sender,
                               IUserContactRepository userRepository) {
        this.sender         = sender;
        this.userRepository = userRepository;
    }

    public void notifyUser(String userId, String message) {
        String contact = userRepository.getContactInfo(userId);
        sender.send(contact, message);
    }
}

// Supporting abstraction for user contact lookup
public interface IUserContactRepository {
    String getContactInfo(String userId);
}

// 4. Unit test — no real email, SMS, or DB needed
@Test
void notifyUser_sendsMessageToCorrectContact() {
    INotificationSender mockSender = Mockito.mock(INotificationSender.class);
    IUserContactRepository mockContacts = Mockito.mock(IUserContactRepository.class);
    when(mockContacts.getContactInfo("user123")).thenReturn("user@example.com");

    NotificationService service = new NotificationService(mockSender, mockContacts);
    service.notifyUser("user123", "Your order is ready");

    verify(mockSender).send("user@example.com", "Your order is ready");
}

// 5. Spring wiring
@Configuration
public class NotificationConfig {
    @Bean
    public INotificationSender notificationSender() {
        return new EmailSender(new EmailClient(smtpConfig()));
    }
}
```

Quick Reference

```
S — One class, one reason to change
O — Extend by adding new classes, not by editing existing ones
L — Subclass must keep every promise the parent made
I — Small focused interfaces beat one fat interface
D — Depend on what something does (interface), not what it is (class)
```